

YO:

Quiero empezar otro proyecto pero partiendo del conocimiento total de lo que está en la carpeta que dice proyecto anterior. En ese sentido vas a leer toda la documentación del proyecto anterior para saber de qué va vas a leer todo el código del index y vas a decirme si esta clara respecto a ese proyecto.

CLAUDE:

Voy a leer toda la documentación y el código del proyecto anterior para entender de qué va.

Ejecutado 2 comandos, leyó 5 archivos

Documentación principal leída. Sigo con los planes OCR, las sesiones documentadas y el backend de Cloud Functions.

Leyó 2 archivos, ejecutado 2 comandos

Ahora leo la sesión restante, el backend de recordatorios y el service worker; después ataco el index.html.

Leyó 2 archivos, ejecutado 6 comandos

Sí — tengo una comprensión clara y completa del proyecto anterior. Leí el CLAUDE.md completo (la documentación maestra), los 4 planes de MD/ (mejoras, comercialización, OCR, pipeline de importación), las 2 sesiones documentadas, el último informe de versión, el backend completo de Cloud Functions, el service worker, los archivos de configuración de Firebase/PWA, y el index.html entero: las ~17.800 líneas de HTML y JavaScript, función por función. Este es el resumen de lo que ahora sé:

Qué es Foresee

Una PWA de gestión de presupuesto personal (mobile-first, en español) con Firebase Auth + Firestore como backend. Un solo usuario real hasta ahora (tú), pero con plan de comercialización en 4 fases (Fase 1 completa; monetización y lanzamiento pendientes).

Versión activa: V FSA 0061 (2026-07-04), con 61 versiones documentadas informe por informe.

Arquitectura

Todo vive en un solo index.html (~620 KB): CSS con tokens de tema claro/oscuro (líneas 38–4554), HTML de las 11 secciones + ~15 modales (4556–6957), y un único `<script type="module">` con ~224 funciones (6957–17781). Sin build, sin tests — verificación manual en navegador.

Estado central `appState` (línea 7031) alimentado por 6 listeners `onSnapshot` de Firestore (`transactions`, `recurringExpenses`, `creditCards`, `projections`, doc de preferencias, doc de gastos comunes), con render reactivo vía `scheduleRenderAll()` (debounce de 50 ms) que solo pinta la pestaña activa.

Ruta de datos en Firestore: `artifacts/wittfinances-282f1/users/{uid}/...`, más docs `archive_YYYY-MM` para el cierre de mes y `fcmTokens` para push.

Backend real (único): Cloud Function programada (`sendRecurringExpenseReminders`, 9:00 AM NY) que barre los recurrentes de todos los usuarios y envía recordatorios por SendGrid (email) y FCM (push) con deduplicación por canal en `emailReminderLog`.

Los mecanismos delicados que ya domino

Saludos y transferencias: la lógica más sutil de la app. `buildInternalPairMap` empareja las dos patas de una transferencia propia (misma fecha+monto, banco distinto, `sharesRefToken` =

últimos 5 caracteres del código de referencia) y decide quién suma y quién resta; buildTransferPairPartners + effectiveOrder mantienen ambas patas adyacentes en la tabla sin romper la cadena de la columna Saldo.

Gastos recurrentes (®): auto-registro mensual con detección de huecos hasta 12 meses atrás, filtro coveredByImport (palabra compartida ± 5 días), cooldown de 60 s en sessionStorage contra parpadeos de auth, y reemplazo por la transacción real al importar (conservando la categoría).

Pipeline de Importar: CSV/Excel + foto (OCR con Tesseract.js lazy-load, dos parsers: desktop de 2 columnas y móvil con fecha-encabezado), cleanBankDescription con su orden crítico de limpieza (la trampa de "ID"/Idaho incluida), duplicados por código $\rightarrow$ texto exacto $\rightarrow$ fecha+monto+banco+palabra compartida, pareo Zelle, batchId para la asignación masiva de banco, snapshot pre-guardado contra falsos duplicados intra-lote. Además leí las 12 "lecciones aprendidas" documentadas para no repetir errores ya pagados.

Voz: parser NLP propio en español (Levenshtein contra bancos/categorías, parser de números escritos, 5 estrategias de extracción de monto).

Cambio de mes: archiva a archive_YYYY-MM sin borrar transacciones (el fix crítico de V FSA 0060 tras la pérdida de datos) + botón de recuperación.

Convenciones del proyecto: nada de innerHTML con datos de usuario, CSS puro sin frameworks, event delegation, todo en un archivo, protocolo de versión/backup/informe/commit con los skills /foresee-find, /foresee-commit, /foresee-mejora-done.

Lo pendiente que quedó registrado

Mejoras 5 (Metas de ahorro) y 6 (Deudas/préstamos) del plan; fases 3–4 de comercialización; deuda técnica declarada: separar el archivo único en módulos y eliminar el patrón window.*.

Única salvedad honesta: del CSS leí los tokens, la estructura de sus ~40 bloques y todo lo que interactúa con JS, pero no memoricé regla por regla las 4.400 líneas de estilos; tampoco abrí uno a uno los 59 backups ni los 61 informes históricos (el historial completo está en CLAUDE.md y leí los relevantes). Nada de eso afecta la comprensión funcional.

Estoy listo para diseñar el proyecto nuevo partiendo de este conocimiento. Cuando quieras, dime qué visión tienes para él (por ejemplo: ¿reescritura modular de Foresee, u otro producto distinto que herede sus lecciones?) y arrancamos.

LA APP SERA: HTML/CSS/JS puro?

Para un proyecto nuevo del tamaño y ambición de Foresee, conviene React (o algo equivalente), y no es una opinión abstracta: la evidencia está en tu propio repositorio:

Tu deuda técnica declarada en CLAUDE.md es exactamente esto. "Archivo único ~600KB — separar CSS/JS/HTML" y "patrón window.* — requiere refactorización mayor". React resuelve ambas de raíz: componentes en archivos separados e imports en vez de variables globales.

Buena parte de tus bugs históricos fueron de sincronización manual del DOM — listeners que se re-adjuntan, tablas que no refrescan, el select anidado que se cerraba solo (V FSA 0057), el `img{display:block}` rompiendo filas. En el modelo declarativo, la mayoría de esa clase de problemas no existe.

La estrategia anti-límite de tokens dejaría de ser necesaria. Hoy cada cambio exige Read con offsets sobre 17.800 líneas y protocolos para no reescribir el archivo. Con componentes de 100–400 líneas, cualquier sesión (mía o tuya) toca solo el archivo relevante.

Tu lógica más valiosa migra casi sin cambios. `cleanBankDescription`, `buildInternalPairMap`, el parser de voz, el pipeline OCR... son funciones puras de JavaScript. En React vivirían en módulos propios (`lib/importacion.js`, etc.), testeables por fin de forma automatizada.

El costo es real pero acotado: hay un paso de build (con Vite es casi invisible: `npm run dev` para trabajar, `npm run build` para publicar) y una curva de aprendizaje de los conceptos de React (estado, props, efectos). Lo puro HTML/JS sigue siendo la opción correcta para páginas pequeñas o prototipos de una tarde — Foresee ya demostró que funciona hasta cierto tamaño, pero también demostró dónde empieza a doler.

YO:

El proyecto de foresee es de un Código muy extenso son más de 17000 líneas de código quiero simplificar el proyecto sin quitar ni una de sus funciones pero en un código más sencillo y pequeño donde después del análisis del código de origen crees todas y cada una de las secciones pero eliminando códigos que estén de más códigos huérfanos, pero quiero seguir con la esencia no quiero librerías excepto las que ya tiene el código, que sea CSS puro sin Tailwind. Y la intención del JSX es que hiero la estética y estilo de esa aplicación quiero comenzar paso a paso y que vayas creando cada sección depurando el código y conservando la funciones y esencia de la app. Dime si está claro y que me recomiendas

CLAUDE:

Sí, está perfectamente claro. Déjame confirmarte lo que entendí y luego te doy mis recomendaciones, porque hay una decisión importante que tomar antes de escribir la primera línea.

Lo que entendí

Reescritura completa de Foresee, no un producto nuevo: las 11 secciones, la voz, el OCR, la importación, los recordatorios — todo se conserva, función por función.

Mismo stack de esencia: HTML + CSS puro + JavaScript vainilla. Sin React, sin Tailwind, sin ninguna librería nueva. Solo las que ya usa la app (Firebase, Chart.js, jsPDF, SheetJS, y Tesseract.js con carga diferida).

El JSX era solo referencia estética: heredar ese estilo — paleta sobria (verdes profundos, piedra, acento ámbar), títulos serif, números en fuente mono alineados, tarjetas limpias con bordes suaves — pero implementado en CSS puro con tokens, como ya hace Foresee con su sistema de variables.

Código depurado: eliminar código huérfano y redundante, simplificar sin perder comportamiento. Del análisis que hice ya tengo candidatos claros: `exportCSV` (los botones están ocultos con `hidden` desde que llegó Excel), íconos duplicados en `PRESET_ICONS`, tokens CSS duplicados (`--color-text-muted` y `--color-text-2` son idénticos), el campo `people` global de gastos comunes que solo sobrevive por una migración vieja, listeners repetidos, y patrones que se repiten decenas de veces (poblar selects, abrir/cerrar modales, confirmaciones) que pueden ser una sola función genérica.

Trabajo incremental: sección por sección, cada una verificada antes de pasar a la siguiente. Mis recomendaciones

1. La decisión clave: archivos separados, aunque sea vainilla puro. Se puede mantener el 100% de la esencia (cero librerías, cero build, cero npm, publica igual en GitHub Pages) pero repartiendo el código con módulos ES nativos, que el navegador soporta directamente:

El único archivo de 17.800 líneas es la razón de casi todos los dolores del proyecto anterior (la "estrategia anti-límite de tokens", los backups completos por cada cambio, el miedo a tocar algo). Con módulos, cada sección vive en un archivo de 300–800 líneas y editarla es trivial. Esta es mi recomendación fuerte. Si aun así prefieres el archivo único porque es parte de la esencia para ti (un solo fichero que copias y funciona), lo respeto y lo hacemos así — la depuración igual puede bajar el total a unas 9.000–11.000 líneas — pero dímelo explícitamente.

2. Mismo Firebase, mismo esquema de datos. El proyecto nuevo debe leer y escribir exactamente las mismas colecciones (artifacts/wittfinances-282f1/users/{uid}/...). Así entras con tu cuenta actual y ves tus datos reales desde el primer día, puedes comparar la app vieja y la nueva lado a lado, y las Cloud Functions de recordatorios siguen funcionando sin tocarlas.

3. Orden de construcción propuesto (de cimientos a complejidad, cada paso usable):

Base — esqueleto HTML, tokens CSS con la nueva estética, tema claro/oscuro, auth completa, appState + listeners de Firestore, navegación y dashboard.

Registros — tabla con saldos, calculadora, categorías, transferencias (aquí migra la lógica de pares de transferencia, la más delicada).

Recurrentes + Proyección (comparten la lógica del auto-registro y ®).

Saldos + Reportes + Presupuesto (los gráficos con Chart.js).

Tarjetas + Gastos comunes.

Configuración completa (exportaciones, cambio de mes, notificaciones, onboarding, tutorial).

Voz.

Importar (CSV/Excel/OCR + escaneo de recibos) — al final por ser lo más complejo, migrando el pipeline con sus 12 lecciones aprendidas intactas.

PWA — manifest, service worker, ícono.

4. Ubicación: el proyecto nuevo va en la raíz del repositorio (por eso movimos todo a proyecto-anterior/), así GitHub Pages servirá la app nueva en la misma URL cuando esté lista.

5. Documentación desde el día uno: un CLAUDE.md nuevo heredando los protocolos que ya te funcionan (versiones, informes, backups), adaptados a la estructura nueva.

Dime dos cosas y arranco con el paso 1: ¿archivos separados con módulos ES (mi recomendación) o un solo index.html como antes?, y ¿confirmas que empiece por la base (auth + tema + dashboard) con la estética del JSX?

YO:

QUIERO EL ARCHIVO SEPARADO HTML, CSS, JS pero antes Quiero que analices línea por línea y el código "origen" así se llamará el archivo base analiza su lógica para que esta nueva app sea lo más depurada posible donde no redunde código innecesarios que todas su

funcionalidad sea la misma y mejor si puedes hacerlo. Desde el punto de vista estético quiero copiar la esencia del archivó jsx quiero contrastes entre los inputs y el fondo en los formularios que los inputs tengan un efecto de fondo que los formularios están mejor diseñados que no hayan formularios excesivos por ejemplo el formulario de registro puede ser el mismo de proyección porque por ejemplo en la app de origen hay secciones que tienen formularios diferentes para la edición y para el registro eso es redundante, usa el estilo del jsx con ese verde pero también usa azul y naranja combínalo de la manera más sobria si que se vea recargado usa los botones de imagen para secciones como la app origen pero dame un concepto diferente para la vista que sea más atractiva has uso de las mejores estrategias de diseño y psicología del color o marketing que atraiga al usuario basando en app del mismo estilo. Esta es una app principalmente para dispositivos móviles por lo tanto hasla responsiva que el texto destaque se vea claramente que sea blanco o negro no uses negrillas o fuente bold para los negativos usa rojo para positivos verde y para saldo azul. Dame tres muestras de la app en modo claro y tres en modo oscuro a ver cuáles elijo para cada modo. Cheque bien el código origen para extraiga de él solo lo que necesito hay muchas líneas de código que por el exceso de edición adonde innecesarias y se pueden simplificar seguramente. Has lo mejor que puedas dame ideas que mejore la app dime si algo está de más o es innecesario establece el mejor plan bien analizado y depurado. Planifiquemos bien antes de ejecutar código.

CLAUDE:

Perfecto, está claro. Ya tengo el análisis línea por línea del código origen hecho en esta misma sesión (leí las 17.784 líneas completas), así que tengo identificados los puntos exactos de redundancia y código huérfano. Antes de escribir el plan final, necesito que decidas dos cosas puntuales que afectan el diseño:

En el código origen, la exportación CSV existe pero está oculta (sus botones tienen `class="hidden"` desde que se agregó la exportación a Excel, que la reemplazó). ¿Qué hago con ella en la app nueva?

Eliminarla (Recomendado)

El estilo del JSX usa una fuente serif para los títulos (le da ese aire institucional/editorial). ¿De dónde tomo la tipografía?

Serif del sistema (Recomendado)

El plan quedó aprobado. Empiezo con la Fase 0: las 6 muestras de diseño. Primero organizo las tareas y luego genero los mockups.

Creado un archivo, usado 12 herramientas

Corrijo dos errores de tipeo que dejé en la paleta A-oscuro antes de ejecutar:

Editado un archivo, ejecutado 2 comandos
Verifico la calidad de las capturas antes de enviártelas:

Leyó 4 archivos, editado un archivo, ejecutado 4 comandos
Las 6 muestras quedaron perfectas. Te las envío para que elijas:
Enviado

Las 6 muestras de Foresee 2.0 — misma pantalla, tres propuestas en modo claro y oscuro

Solicitado

Modo claro

Denegado

Perfecto — respóndeme por aquí directamente cuando las hayas visto: dime cuál propuesta quieres para el modo claro y cuál para el modo oscuro (pueden ser distintas, por ejemplo "claro C, oscuro B").

Si ninguna te convence del todo, también puedes pedirme ajustes sobre una de ellas ("la A pero con el botón naranja de la B", "la C con cabecera verde", etc.) y te genero la variante corregida antes de congelar los tokens y arrancar la Fase 1.

YO:

Quiero los formularios como modales flotantes y aparezcan al presionar un botón como la app origen que no ocupen espacio en el Dashboard, que las etiquetas de saldo ingreso y egreso tengan los colores respectivos a su operación rojo gasto, verde ingreso, azul saldo. Menos que sea negativo los números. Serían rojos esos colores pueden ser el fondo u pestañas como bordes

CLAUDE:

Entendido — son tres ajustes de diseño claros. Actualizo el plan de las muestras para reflejarlos y te presento la versión corregida del Paso 0.

modal flotante abierto. Tarjetas con color por operación y negativos siempre en rojo.

Listo — ya tienes en el panel las 6 muestras de la ronda 2 con tus tres ajustes aplicados:

Formulario como modal flotante: el dashboard quedó limpio, solo con los botones "+ Registrar" (naranja) y "⇌ Transferir" (azul). La segunda vista de cada muestra enseña el modal abierto flotando sobre el dashboard oscurecido y desenfocado, igual que funcionará en la app.

Tarjetas con el color de su operación: Saldo con fondo azulado + pestaña azul, Ingresos con fondo verdoso + pestaña verde, Gastos con fondo rojizo + pestaña roja — incluso las etiquetas ("Saldo actual", "Ingresos del mes"...) van en su color.

El signo manda: en la tabla incluí a propósito un saldo negativo (-45,40) para que veas la regla en acción — se pinta rojo aunque la columna Saldo sea azul.

Las tres propuestas siguen siendo:

A · Esmeralda — cabecera verde profundo, botón ámbar (la del espíritu del JSX)

B · Azul financiero — cabecera azul profundo, botón naranja

C · Piedra cálida — sin cabecera de color, botón verde, la más minimalista

Cuando las veas, dime cuál eliges para modo claro y cuál para modo oscuro (pueden ser distintas), o pídemme ajustes sobre cualquiera. Con tu elección congelo los tokens en `css/base.css` y arranco la Fase 1 (esqueleto + auth + dashboard real conectado a tus datos).

yo:

Quiero los formularios tipo calculadora como el de origen con el mismo comportamiento y recuerda que debe haber un modal de categoría que permita seleccionar la categoría

claude:

Plan propuesto

Foresee 2.0 — Reescritura depurada del código origen

Contexto

El proyecto Foresee vive en ``proyecto-anterior/index.html`` (**el "origen"**: 17.784 líneas — CSS, HTML y JS en un solo archivo, V FSA 0061). Funciona, pero el archivo único genera todos los dolores documentados en su propio CLAUDE.md: ediciones riesgosas, backups completos por cambio, protocolos anti-límite de tokens, y ~60 versiones de parches acumulados que dejaron código huérfano y lógica duplicada

****Objetivo:**** reconstruir la app completa en la raíz del repositorio con ****exactamente la misma funcionalidad****, en código depurado y modular (módulos ES nativos, sin build, sin librerías nuevas, CSS puro sin Tailwind), con una estética nueva inspirada en el JSX contable que compartió el usuario (verde profundo + azul + naranja, sobrio), mobile-first, conectada a ****las mismas colecciones de Firestore**** (los datos reales del usuario funcionan desde el día 1). Rama de trabajo: ``claude/repo-restructure-subfolder-h7ur3w``. El origen NO se toca — queda en ``proyecto-anterior/`` como referencia.

Decisiones ya tomadas con el usuario

| Decisión | Elección |

|---|---|

| Arquitectura | Archivos separados con módulos ES nativos (sin build, publica igual en GitHub Pages) |

| Librerías | Solo las del origen: Firebase 11, Chart.js 4.4.4, jsPDF + autotable, SheetJS, Tesseract.js 5 (lazy) |

| CSS | Puro, con tokens/variables. Sin Tailwind ni frameworks |

| Estética | Esencia del JSX: verde esmeralda profundo + azul + naranja/ámbar, sobrio, sin recargar |

| Tipografía | Serif del sistema (Georgia/serif) para títulos; sans del sistema para el resto. Sin Google Fonts |

| Export CSV | **Se elimina** (ya estaba oculta en el origen; Excel la reemplazó) |

| Colores de montos | Negativos rojo, positivos verde, saldos azul. **Sin bold** — el énfasis lo da el color y la fuente mono tabular. Texto principal blanco/negro puro según tema |

| Formularios | **Unificados**: un solo formulario para crear Y editar (el origen tiene modales separados). El de transacción y el de proyección son el mismo con un parámetro de modo |

| Navegación | Botones de sección con imagen (los mismos íconos Cloudinary del origen) pero con un concepto visual nuevo |

| Datos | Mismo proyecto Firebase, mismas rutas
`artifacts/wittfinances-282f1/users/{uid}/...` — compatibilidad total con la app vieja y con las Cloud Functions existentes |

PASO 0 — Muestras de diseño (primera entrega, antes de construir)

> **Revisión solicitada por el usuario tras la primera ronda de muestras:**

> 1. **Formularios como modales flotantes** — como en la app origen: se abren al presionar un botón y NO ocupan espacio en el dashboard. Cada muestra enseña dos vistas lado a lado: (a) el dashboard con sus botones de acción (+ Registrar, ⇌ Transferir) y (b) el modal flotante abierto sobre el fondo oscurecido.

> 2. **Tarjetas del dashboard codificadas por color de operación**: la tarjeta de Saldo lleva azul, la de Ingresos verde y la de Gastos rojo — aplicado como fondo tintado y/o pestaña-borde lateral de color, no solo en la cifra.

> 3. **Los negativos siempre en rojo**: si el saldo es negativo, su número se muestra rojo aunque la tarjeta sea azul. La regla de color de la cifra la manda el signo; la tarjeta identifica la operación.

Crear **6 mockups HTML estáticos** (autocontenidos, sin JS de app) de una pantalla representativa (dashboard con tarjetas codificadas por color + tabla de registros + modal flotante de nueva transacción abierto) para que el usuario elija:

- **Propuesta A — "Esmeralda institucional"**: la más fiel al JSX. Header verde-950, fondo piedra, acento ámbar en CTAs, azul reservado para saldos. (claro + oscuro)

- **Propuesta B — "Azul financiero"**: azul profundo como color de marca (continuidad con el Foresee actual), verde solo para positivos, naranja para acciones/alertas. (claro + oscuro)

- **Propuesta C — "Piedra cálida"**: fondo neutro dominante, tarjetas muy limpias, los tres acentos (verde/azul/naranja) solo en datos y botones — el más minimalista. (claro + oscuro)

Requisitos comunes a las 3: inputs con **contraste real contra el fondo del formulario** (fondo tintado + sombra interior + borde de color al foco), números en mono tabular, tarjetas con radios suaves, mobile-first. **El formulario aparece únicamente como modal flotante** (botón "+ Registrar" en el dashboard lo abre); las tarjetas de Saldo/Ingresos/Gastos llevan su color de operación como fondo

tintado + borde-pestaña lateral (azul/verde/rojo); los números negativos siempre en rojo sin importar la tarjeta. Se entregan como imágenes con dos vistas por muestra (dashboard y modal abierto) para que el usuario elija **una para modo claro y una para modo oscuro** (pueden ser propuestas distintas). Los tokens elegidos se congelan en `css/base.css`.

Este mismo criterio queda como regla de la app definitiva: **ningún formulario vive inline en una sección** — todos son modales flotantes disparados por botones de acción, igual que en la app origen.

—

Arquitectura de archivos``

/ (raíz del repo — la app nueva)

- |— index.html # solo estructura: secciones, modales, nav (~900 líneas)
- |— manifest.json # igual al origen (mismo ícono Cloudinary)
- |— sw.js # igual al origen (cache + push FCM)
- |— .nojekyll
- |— css/
 - | |— base.css # tokens (2 temas), reset, tipografía, botones, inputs, modales, tablas
 - | |— secciones.css # estilos específicos por sección
- |— js/
 - |— main.js # arranque: Firebase init, onAuthStateChanged, wiring global
 - |— state.js # appState + suscripciones Firestore + scheduler de render
 - |— firebase.js # init + re-export de las funciones del SDK (único punto de import del CDN)

```

└── lib/
    ├── utils.js    # fmt moneda/fecha, toast, toTitleCase, withinDays,
sharesDescriptionWord...
    ├── ui.js       # gestor genérico de modales
(abrir/cerrar/overlay/focus-trap), poblar selects, confirmación, tooltip
    ├── balances.js # MOTOR ÚNICO de saldos: pares de transferencia,
orden efectivo, saldo corrido
    ├── recurrentes-engine.js # auto-registro ®, coveredByImport, cooldown,
recordatorios
    ├── import-pipeline.js # CSV/XLSX, cleanBankDescription, duplicados,
pairZelle
    ├── ocr.js       # Tesseract lazy, cluster/split/parsers desktop+móvil,
recibos
    └── voz-parser.js # numberParser, diccionarios, parseVoiceInput
└── secciones/
    ├── dashboard.js, registros.js, proyeccion.js, recurrentes.js,
    ├── comunes.js, tarjetas.js, saldos.js, reportes.js, presupuesto.js,
    └── configuracion.js, voz.js, importar.js

```

``Objetivo de tamaño: **~9.000–10.500 líneas totales** (vs 17.784), sin perder una sola función visible.

Depuración — qué se elimina o unifica (hallazgos del análisis línea por línea del origen)

Código muerto / huérfano (se elimina)

1. **exportCSV()** completa + 3 botones ocultos (``cfg-export-month-btn``, ``cfg-export-all-btn``, ``btn-export-csv-month``) — origen líneas ~9596-9666. Decidido con el usuario.
2. **Migración de ``gastosComunes.people[]`` global** (origen ~7691-7697): formato viejo que ya no existe en los datos. La app nueva asume ``items[].people``.
3. **Tokens CSS duplicados**: ``--color-text-2`` $\equiv$ ``--color-text-muted``, ``--color-primary`` $\equiv$ ``--color-accent``, ``--color-bg-secondary`` $\equiv$ ``--color-surface``, ``--color-bg-tertiary`` $\equiv$ ``--color-surface-2``, ``--color-input`` $\equiv$ ``--color-surface``. La paleta nueva se define una sola vez.
4. **PRESET_ICONS** con entradas duplicadas (ej. ``deudas_jcqhez.png`` aparece 2 veces, origen ~7114 y ~7124).
5. **Alias ``updateDeleteButtonVisibility = updateSelectionButtons``** (origen ~8677) — un solo nombre.
6. **``localStorage.setItem("foresee-filter-month")`` se escribe en 3 lugares pero nunca se lee** (huérfano). Decisión: convertirlo en mejora — restaurar el mes filtrado al abrir la app (ver Mejoras).
7. Restos varios: comentario duplicado ``/* === */`` doble en línea ~9075-9076, listeners de tipo de voz con 8 llamadas `classList` que se reducen a 2, ``ICON_TRASH_SOLID`` usado una sola vez.

Redundancias que se unifican (misma funcionalidad, menos código)

8. **Formulario único de movimiento** — el origen tiene: modal calculadora (crear/editar transacción), modal proyección (``proj-modal``, crear/editar), y ``finalizeTransaction``/``finalizeProjection`` casi idénticas (~90 líneas duplicadas). La app nueva tiene UN formulario de movimiento con modos (``transaccion | proyeccion``) y UNA función de guardado parametrizada por colección. **Decisión del usuario (Fase 0, ronda 3): el formulario ES la calculadora del origen, con el mismo comportamiento exacto** — barra de tipo Ingreso/Gasto, fecha → descripción → banco, teclado numérico con operadores (+-x÷, ☒, `safeEval`), botón «✓ Listo», y al confirmar se abre el **modal de selección de**

categoría** (grid de categorías con sus íconos) que completa el guardado. El modal de categoría se reutiliza también para editar la categoría desde Registros y para asignar categoría en Importar, igual que en el origen.

9. ****Motor único de saldos (`lib/balances.js`)**** — hoy la lógica de "qué suma y qué resta" está repetida con variantes en 5 sitios: ``renderTransactionsTable``, ``renderDashboard``, ``exportExcel``, ``renderProyeccionTable``, ``renderBankBalances``. Además tienen ****inconsistencias reales entre sí**** (ver Mejoras #2). Una sola función ``computeBalances(transactions)`` devuelve pares, roles, orden efectivo y mapa de saldos; todos consumen eso.

10. ****Poblado de selects**** — ``populateBankSelects``, ``populateCategorySelect`` + 6 reconstrucciones inline (rec-modal, proj-modal, shared-bank, voice, filtros) → una utilidad ``fillSelect(el, items, placeholder)``.

11. ****Gestor de modales genérico**** — el patrón abrir/cerrar/click-overlay/focus-trap/aria-hidden está repetido ~15 veces + un `MutationObserver`. Un módulo ``ui.js`` con ``openModal(id)/closeModal(id)`` y delegación única.

12. ****Fila expandible "info"***** — el bloque de crear ``info-row`` está copiado idéntico en los listeners de ``reg-tbody`` y ``proj-tbody`` (~30 líneas x2) → una función.

13. ****"¿Guardar descripción?"***** — el prompt + `setDoc` está duplicado en ``finalizeTransaction`` y ``finalizeProjection`` → utilidad única (y ``handleRecFormSubmit`` usa una tercera variante, se unifica).

14. ****Actualización en vivo de Presupuesto**** — el listener ``input`` de ``budget-tbody`` (origen ~14136-14221) re-implementa el cálculo de ``renderBudgetTable`` (~85 líneas). Se reemplaza por recalcular la fila con la misma función de render.

15. ****Enter-flow de formularios**** — las cadenas de ``keydown Enter`` → focus siguiente repetidas para calc/rec/cc → helper ``enterFlow([ids])``.

16. ****`showCatDetail` y el onclick del donut**** se redefinen en cada ``renderReportes()`` → se definen una vez.

17. **CSS reescrito desde cero** (~4.500 líneas → objetivo ~1.600): el origen arrastra selectores de skins anteriores y overrides acumulados de 60 versiones. Con la estética nueva no se migra el CSS, se escribe limpio sobre los tokens elegidos en el Paso 0.

Lo que se copia casi intacto (lógica sensible ya depurada por iteración real — NO reinventar)

- `cleanBankDescription`, `extractImportCode`, `detectImportDuplicates`, `pairZelleRows`, `sharesRefToken`, `sharesDescriptionWord` (con sus 12 lecciones aprendidas del doc de referencia).
- Todo el pipeline OCR (parsers desktop/móvil, stripDividerLines, regex de fechas/montos, modo debug `FORESEE\_OCR\_DEBUG`) y el escáner de recibos.
- `processRecurringExpenses` (detección de huecos 12 meses, cooldown 60s) y la transición de mes SIN borrado (fix V FSA 0060) + recuperación de archivo.
- `parseVoiceInput` completo con sus diccionarios.
- `calculateMonthlyPayment`, `computeAllAmounts` (distribución de gastos comunes en cascada).
- `sw.js`, `manifest.json`, `functions/` (no se tocan — siguen sirviendo a ambas apps).

Estética (reglas congeladas, a materializar en el Paso 0)

- **Paleta tri-color sobria**: verde esmeralda (marca/positivos), azul (saldos/información), naranja/ámbar (CTA principal y alertas suaves), rojo solo para negativos y peligro. Regla 60/30/10: neutros dominan, un solo acento fuerte por pantalla.
- **Semántica de montos**: gasto/negativo = rojo, ingreso/positivo = verde, saldo = azul. Sin `font-weight: bold` en cifras — mono tabular + color.
- **Texto**: contraste AA mínimo; texto principal negro puro (claro) / blanco puro (oscuro), como pidió el usuario.

- ****Inputs con presencia****: fondo tintado distinto al de la tarjeta, sombra interior sutil, borde que se enciende (color de acento) al foco — el efecto que el usuario señaló del JSX.
- ****Tarjetas****: bordes 1px suaves, radio 12-16px, sombra mínima; headers de sección tipo "cinta" con label uppercase pequeño (como el libro fiscal del JSX).
- ****Títulos serif del sistema****, cuerpo sans del sistema.
- ****Nav****: botones con imagen (íconos Cloundinary existentes) en un concepto nuevo — barra inferior fija en móvil (patrón de apps financieras: pulgar-alcanzable) con "Más" para secciones secundarias, y rail lateral en desktop. El dashboard con las 3 tarjetas grandes se mantiene como home.
- ****Psicología de color aplicada****: verde = confianza/dinero sano, azul = estabilidad (saldos), naranja = acción sin agresividad; rojo reservado exclusivamente a pérdida/peligro para que no pierda significado.

Mejoras propuestas (mismas funciones, mejor comportamiento)

1. ****Restaurar el mes filtrado**** al abrir la app (arregla el huérfano #6: hoy se guarda y nunca se lee).
2. ****Unificar el cálculo de saldo del dashboard**** con el motor de balances: en el origen, el saldo del dashboard ignora las transferencias externas (Zelle in/out con `transferDirection`) y el gráfico "Evolución del Saldo" las resta todas — tres pantallas pueden mostrar tres saldos distintos. Con `lib/balances.js` todas las vistas dicen lo mismo.
3. ****Estados vacíos con guía****: cada sección sin datos muestra qué hacer ("Agrega tu primer banco en Configuración →") en lugar de solo texto plano.
4. ****Confirmaciones consistentes**** y toasts con la misma voz en toda la app (hoy hay variantes de redacción acumuladas).
5. ****Accesibilidad****: roles/aria ya presentes en el origen se conservan y se completan (labels reales en todos los inputs).

6. Opcionales a futuro (NO en esta reescritura, solo se dejan anotados):
búsqueda de texto en Registros, ingresos recurrentes, IndexedDB offline real —
están en la lista "Tareas posteriores" del plan de mejoras del origen.

Fases de ejecución (cada una termina usable, verificada y con commit)

| Fase | Contenido | Verificación contra el origen |

|---|---|---|

| ****0**** | 6 mockups de diseño (3 claro / 3 oscuro) → usuario elige | Visual,
elección del usuario |

| ****1**** | Esqueleto: index.html + tokens CSS del diseño elegido + tema
claro/oscuro + auth completa (login/registro/recuperar/eliminar cuenta) +
`state.js` con los 6 listeners Firestore + nav + dashboard con alertas | Login con
cuenta real; dashboard muestra los mismos números que el origen |

| ****2**** | ****Registros****: tabla con saldo corrido (motor de balances), formulario
unificado con calculadora, categorías, edición inline de descripción/categoría,
selección múltiple, transferencias, badges ⚠ duplicados, filas info |
Comparación fila a fila y saldo a saldo con el origen abierto al lado |

| ****3**** | ****Recurrentes + Proyección****: CRUD, auto-registro ®, proyección
dinámica con virtuales + mes siguiente, recordatorios de navegador | Mismos
asientos virtuales y mismo saldo proyectado que el origen |

| ****4**** | ****SalDOS + Reportes + Presupuesto****: cards por banco, 7 bloques de
reportes con Chart.js, presupuesto con barras y autosave | Mismos totales por
banco/categoría |

| ****5**** | ****Tarjetas + Gastos comunes****: cards editables, distribución
equal/%/manual, registro de pagos, revertir | Mismos montos de distribución |

| ****6**** | ****Configuración completa****: categorías con íconos, bancos,
descripciones, alias, saldo inicial, divisa, contraseña, toggles de
notificaciones/email/push (FCM), JSON import/export, Excel, PDF, nuevo mes,
recuperar archivo, reiniciar período, eliminar cuenta, onboarding + demo +
tutorial de videos | Cada acción probada contra Firestore |

| ****7**** | ****Voz****: parser completo + panel de confirmación | Frases de ejemplo del origen parsean igual |

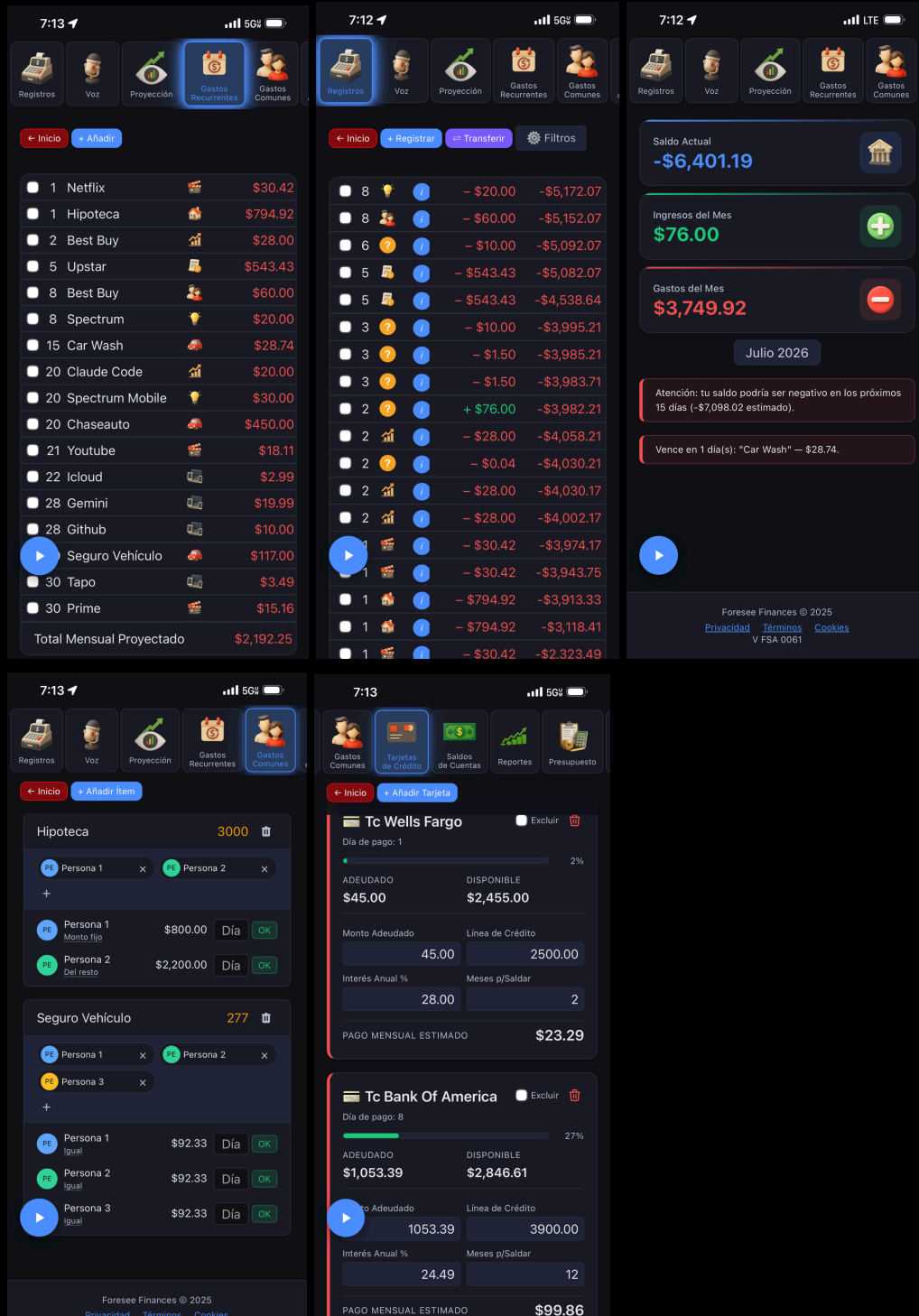
| ****8**** | ****Importar****: CSV/XLSX + OCR foto + escáner de recibos, tabla preview completa (banco por fila/lote, needsReview, duplicados) | CSVs y capturas de prueba; `node --check` sobre módulos |

| ****9**** | ****PWA + cierre****: manifest, sw, fullscreen, pull-to-refresh, swipe entre tabs, offline dot; `CLAUDE.md` nuevo con protocolos adaptados; commit final y push | Lighthouse PWA instalable; checklist funcional completo |

En cada fase: `python3 -m http.server` para probar en local (los módulos ES requieren servidor), verificación funcional real contra la cuenta del usuario, y `node --check` de cada módulo JS. Commit por fase con mensaje descriptivo; push a la rama designada.

yo:

Lo que quiero es mantener el mismo estilo en las secciones y el Dashboard tal cual se ven en la imagen tal vez mover los botones del Tab-Btn a la parte de abajo y pegarlos a margen inferior de la pantalla



Checklist maestro de paridad funcional

Se mantiene un archivo `MD/paridad-funcional.md` en el proyecto nuevo con las ~90 funciones visibles del origen (extraídas del análisis) marcándose  a medida que cada una queda verificada — garantía de "ni una función menos".

El estilo de las secciones y el Dashboard se mantiene TAL CUAL se ve en tus capturas — mismo tema oscuro azul-marino, mismos íconos 3D, mismas tarjetas con su línea de color (azul saldo, verde ingresos, rojo gastos), mismos botones de acción (rojo «← Inicio», azul «+ Registrar», púrpura «⇌ Transferir»), mismos acentos por sección, y el tema claro "Gris Perla" que ya existe.

Único cambio de layout: los botones del tab-btn (con sus imágenes 3D) se mueven abajo, pegados y fijos al margen inferior de la pantalla, scrolleables horizontalmente igual que hoy.

Nada más se rediseña. Las muestras A/B/C quedaron descartadas.

No hay nada pendiente de decidir — el plan está completo y el diseño cerrado. Para arrancar la construcción tienes dos caminos equivalentes:

En esta sesión: escríbeme "Ejecuta la Fase 1 completa" (y si aparece un diálogo de aprobación del plan, acéptalo).

En una sesión nueva (si esta llega al límite): primero pídele que haga checkout de la rama `claude/repo-restructure-subfolder-h7ur3w` y luego "Continúa Foresee 2.0 según el plan" — el CLAUDE.md le dará todo el contexto.